Week 7 : Binary Trees and Heaps Objectives

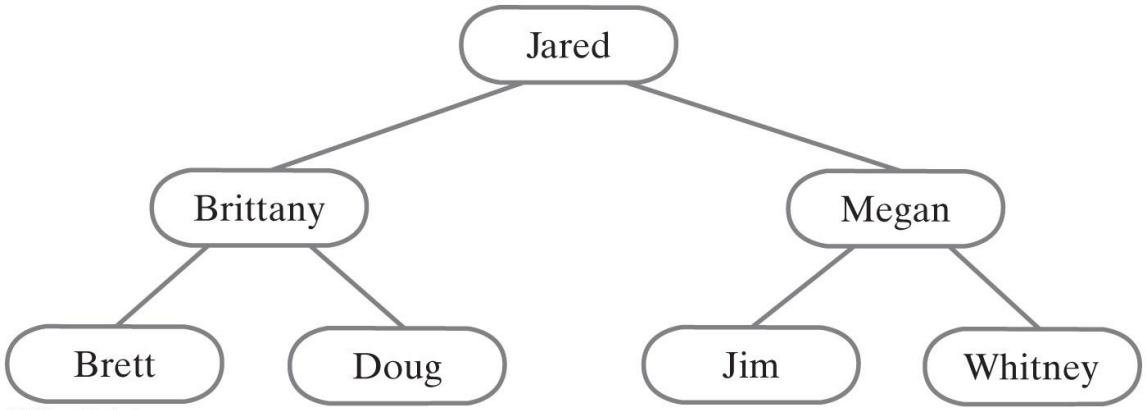
Upon completion of this Binary tree and heaps unit, students will be able to understand and explain the following sub topics :

- A Binary Search Tree Implementation
- Searching and Retrieving
- Traversing
- Adding an Entry
- A Recursive Implementation
- An Iterative Implementation
- Removing an Entry
- Removing an Entry Whose Node Is a Leaf , Has One Child , Two Children , and removing an Entry in the Root
- A Recursive Implementation , An Iterative Implementation
- The Efficiency of Operations
- The Importance of Balance
- The Order in Which Nodes Are Added
- An Implementation of the ADT Dictionary

- A Heap Implementation
- Reprise: The ADT Heap
- Using an Array to Represent a Heap
- Adding an Entry
- Removing the Root

Binary Search Tree

- For each node in a binary search tree
  - Node's data is greater than all data in node's left subtree
  - Node's data is less than all data in node's right subtree
- Every node in a binary search tree is the root of a binary search tree



© 2019 Pearson Education, Inc.

Interface for the Binary Search Tree (Part 1)

```
/** An interface for a search tree. */
public interface SearchTreeInterface<T extends Comparable<? super T>>
      extends TreeInterface<T>
{
   /** Searches for a specific entry in this tree.
      @param anEntry  An object to be found.
      @return  True if the object was found in the tree. */
   public boolean contains(T anEntry);

   /** Retrieves a specific entry in this tree.
      @param anEntry  An object to be found.
      @return  Either the object that was found in the tree or
               null if no such object exists. */
   public T getEntry(T anEntry);

   /** Adds a new entry to this tree, if it does not match an existing
      object in the tree. Otherwise, replaces the existing object with
      the new entry.
      @param anEntry  An object to be added to the tree.
      @return  Either null if anEntry was not in the tree but has been added, or
               the existing entry that matched the parameter anEntry
```

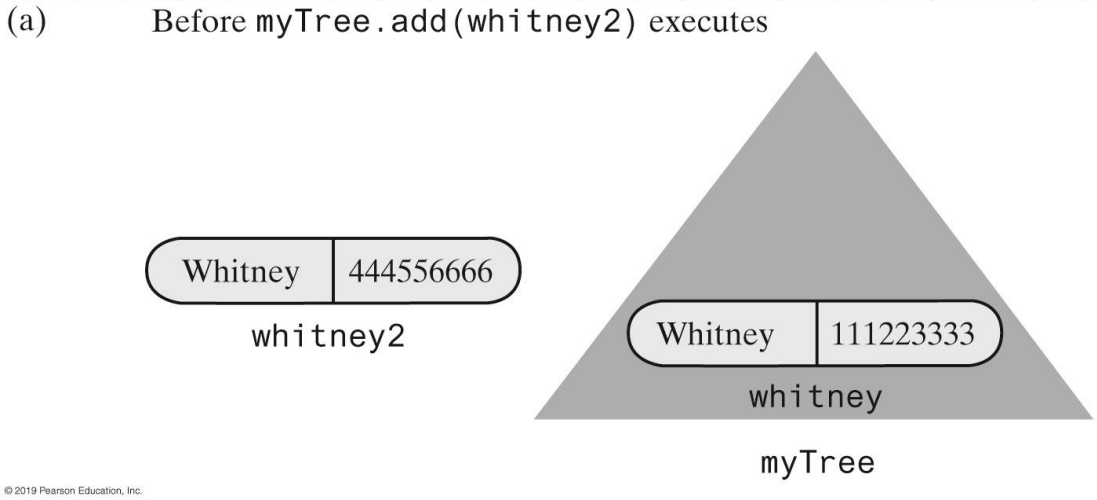
Interface for the Binary Search Tree (Part 2)

```
/** Removes a specific entry from this tree.
   @param anEntry  An object to be removed.
   @return  Either the object that was removed from the tree or
            null if no such object exists. */
public T remove(T anEntry);

/** Creates an iterator that traverses all entries in this tree.
   @return  An iterator that provides sequential and ordered access
            to the entries in the tree. */
public Iterator<T> getInorderIterator();
} // end SearchTreeInterface
```

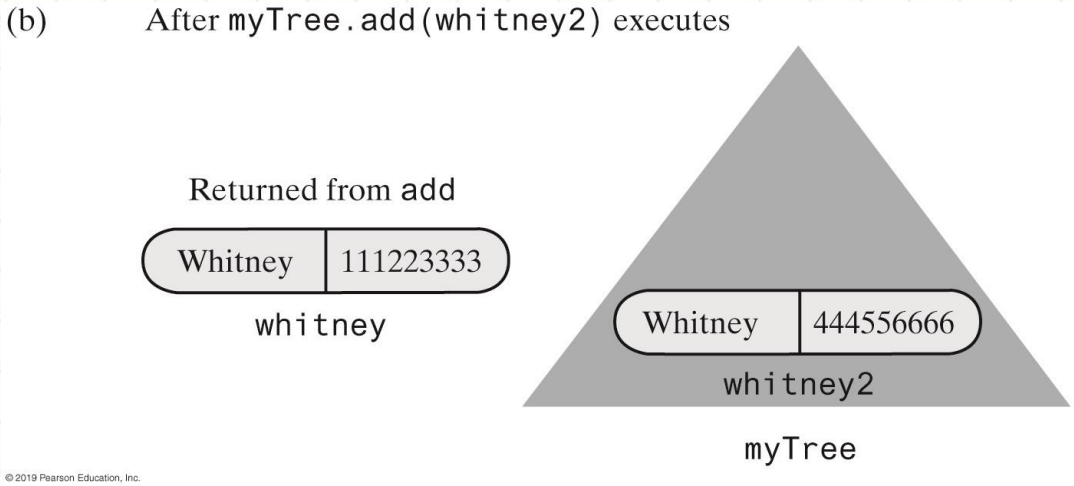
Understanding the Specifications

- Methods will use return values instead of exceptions to indicate whether an operation has failed
  - **`getEntry`**, returns same entry it is given to find
  - **`getEntry`** returns an object in tree and matches given entry according to the entry's **`compareTo`** method

Adding to a Binary Search Tree



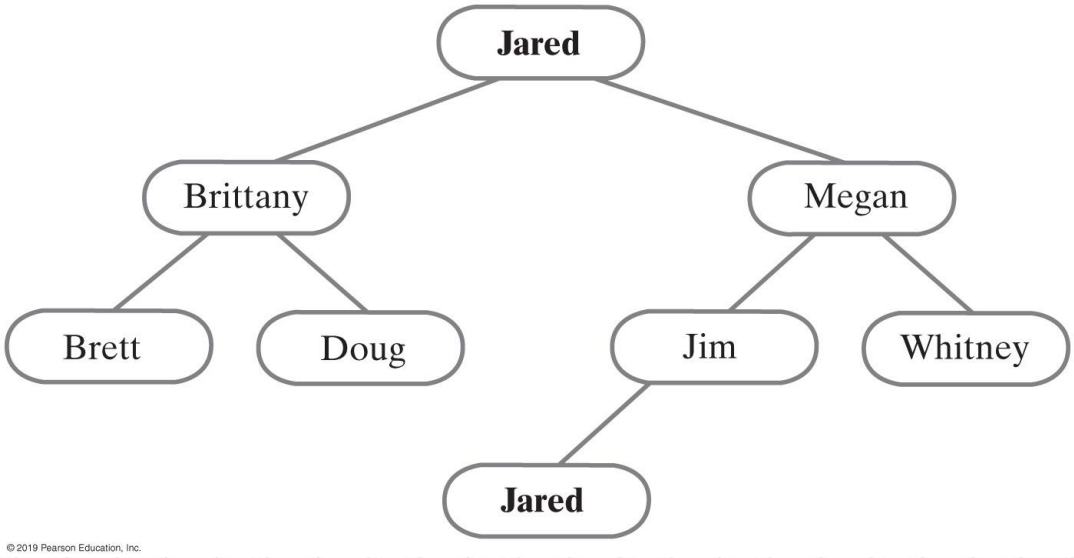
© 2019 Pearson Education, Inc.



© 2019 Pearson Education, Inc.

Duplicate Entries

- If any entry $\boldsymbol{e}$ has a duplicate entry $\boldsymbol{d}$, we arbitrarily require that $\boldsymbol{d}$ occur in the right subtree of $\boldsymbol{e}$'s node
- For each node in a binary search tree:
  - Data in a node is greater than data in node's left subtree
  - Data in a node is less than or equal to data in node's right subtree



Beginning the Class Definition

```
package TreePackage;
import java.util.Iterator;
/** A class that implements ADT binary search tree by extending BinaryTree. */
public class BinarySearchTree<T extends Comparable<? super T>>
            extends BinaryTree<T> implements SearchTreeInterface<T>
{
   public BinarySearchTree()
   {
      super();
   } // end default constructor

   public BinarySearchTree(T rootEntry)
   {
      super();
      setRootNode(new BinaryNode<T>(rootEntry));
   } // end constructor

   // Disable setTree (see Segment 26.6)
   public void setTree(T rootData, BinaryTreeInterface<T> leftTree,
                              BinaryTreeInterface<T> rightTree)
   {
      throw new UnsupportedOperationException();
```

Searching and Retrieving

- Recursive algorithm to search a binary search tree

```
Algorithm bstSearch(binarySearchTree, desiredObject)
// Searches a binary search tree for a given object.
// Returns true if the object is found.
if (binarySearchTree is empty)
   return false
else if (desiredObject == object in the root of binarySearchTree)
   return true
else if (desiredObject < object in the root of binarySearchTree)
   return bstSearch(left subtree of binarySearchTree, desiredObject)
else
   return bstSearch(right subtree of binarySearchTree, desiredObject)
```

Searching and Retrieving

- Algorithm that describes actual implementation more closely

```
Algorithm bstSearch(binarySearchTreeRoot, desiredObject)
// Searches a binary search tree for a given object.
//Returns true if the object is found.
if (binarySearchTreeRoot is null)
        return false
else if (desiredObject == object in binarySearchTreeRoot)
        return true
else if (desiredObject < object in binarySearchTreeRoot)
        return bstSearch(left child of binarySearchTreeRoot, desiredObject)
else
        return bstSearch(right child of binarySearchTreeRoot, desiredObject)
```

Searching and Retrieving

- The method `getEntry` uses `findEntry`

```
public T getEntry(T anEntry)
{
   return findEntry(getRootNode(), anEntry);
} // end getEntry

private T findEntry(BinaryNode<T> rootNode, T anEntry)
{
   T result = null;

   if (rootNode != null)
   {
      T rootEntry = rootNode.getData();

      if (anEntry.equals(rootEntry))
         result = rootEntry;
      else if (anEntry.compareTo(rootEntry) < 0)
         result = findEntry(rootNode.getLeftChild(), anEntry);
      else
         result = findEntry(rootNode.getRightChild(), anEntry);
   } // end if
```

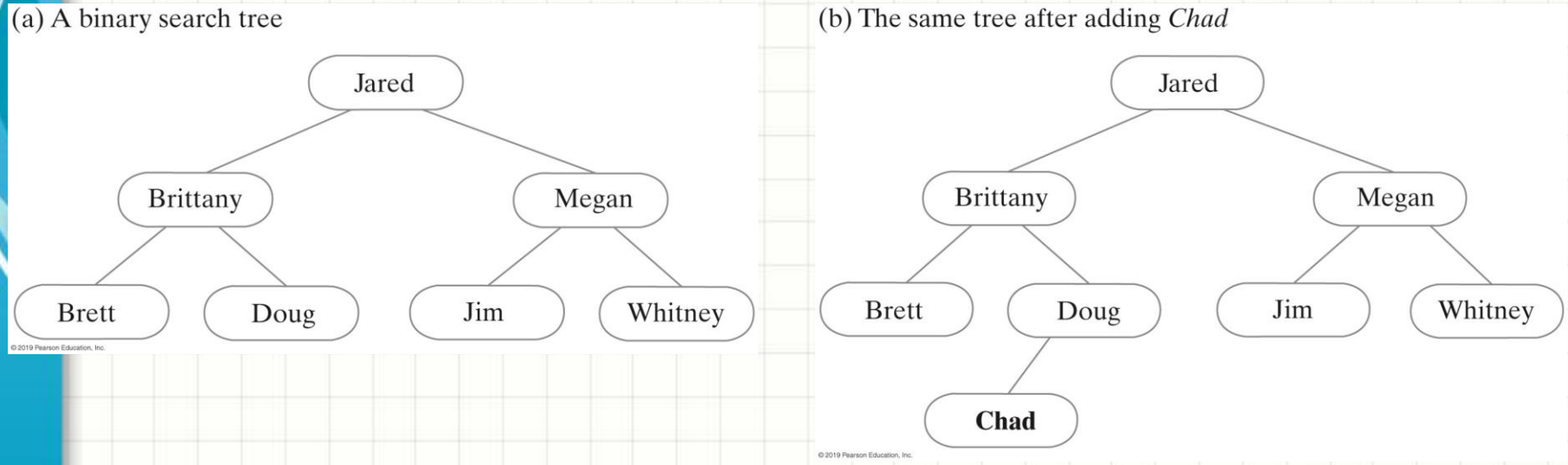
Searching and Retrieving

```
public boolean contains(T anEntry)
{
   return getEntry(anEntry) != null;
} // end contains
```

- Method `contains` can simply call `getEntry` to see whether a given entry is in the tree

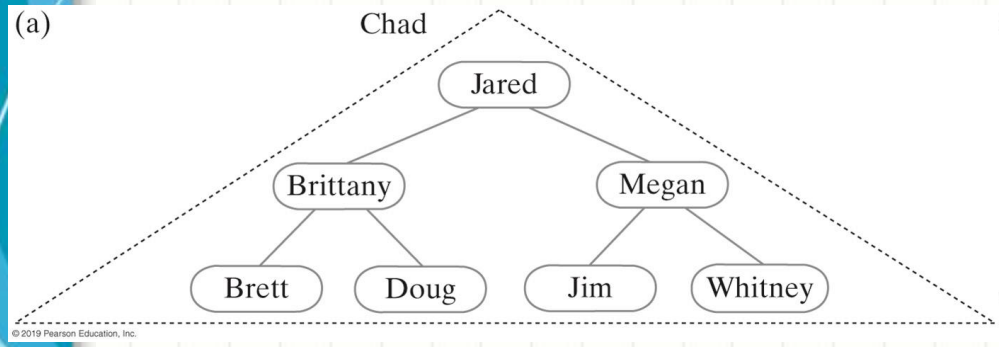
Adding to a Binary Search Tree

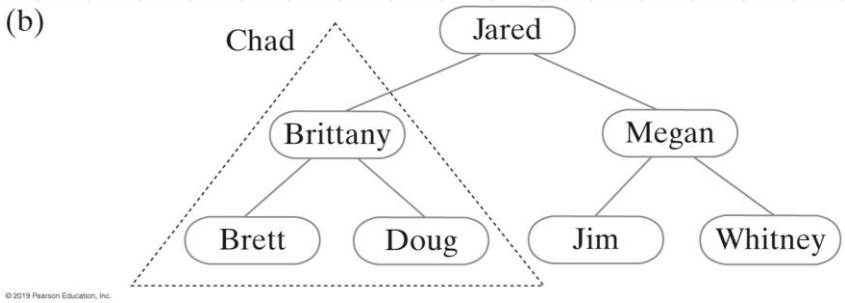
- FIGURE 26-4 A binary search tree before and after adding Chad

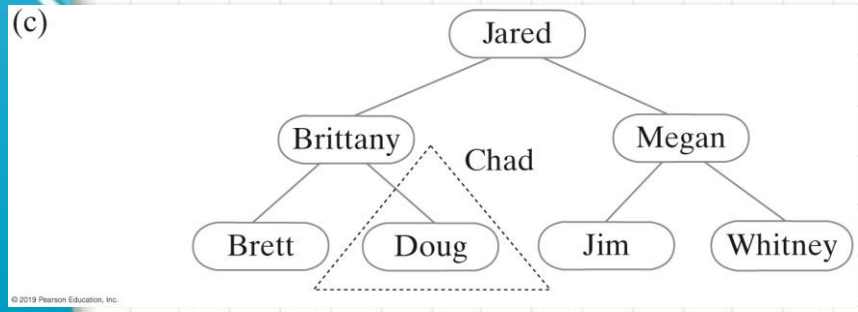


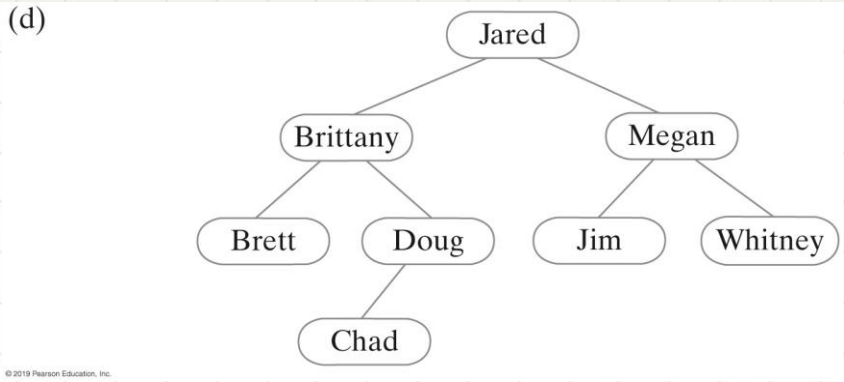
Adding to a Binary Search Tree

- Recursively adding Chad to smaller subtrees of a binary search tree









Recursive Add Implementation

- Recursive algorithm for adding a new entry

```
Algorithm addEntry(binarySearchTree, anEntry)
// Adds an entry to a binary search tree that is not empty.
//  Returns null if anEntry did not exist already in the tree. Otherwise, returns the
//tree entry that matched and was replaced by anEntry.
result = null
if (anEntry matches the entry in the root of binarySearchTree)
{
        result = entry in the root
        Replace entry in the root with anEntry
}
else if (anEntry < entry in the root of binarySearchTree)
{
        if (the root of binarySearchTree has a left child)
                result = addEntry(left subtree of binarySearchTree, anEntry)
        else
Give the root a left child containing anEntry
}
```

Recursive Add Implementation

- Recursive algorithm for adding a new entry

```
Algorithm addEntry(binarySearchTree, anEntry)
// Adds an entry to a binary search tree that is not empty.
//  Returns null if anEntry did not exist already in the tree. Otherwise, returns the
//tree entry that matched and was replaced by anEntry.
result = null
if (anEntry matches the entry in the root of binarySearchTree)
{
        result = entry in the root
        Replace entry in the root with anEntry
        }
else if (anEntry < entry in the root of binarySearchTree)
{
        if (the root of binarySearchTree has a left child)
                result = addEntry(left subtree of binarySearchTree, anEntry)
        else
        Give the root a left child containing anEntry
}
else // anEntry > entry in the root of binarySearchTree
{
```

Recursive Implementation

- Handle the addition to an empty binary search tree as a special case

```
Algorithm add(binarySearchTree, anEntry)
// Adds an entry to a binary search tree.
// Returns null if anEntry did not exist already in the tree. Otherwise, returns the
// tree entry that matched and was replaced by anEntry.
result = null
if (binarySearchTree is empty)
        Create a node containing anEntry and make it the root of binarySearchTree
else
        result = addEntry(binarySearchTree, anEntry)
return result;
```

Recursive Implementation

- The public method `add`

```
public T add(T anEntry)
{
   T result = null;

   if (isEmpty())
      setRootNode(new BinaryNode<>(anEntry));
   else
      result = addEntry(getRootNode(), anEntry);

   return result;
} // end add
```

Recursive Implementation — method `addEntry`

```
// Adds anEntry to the nonempty subtree rooted at rootNode.
private T addEntry(BinaryNode<T> rootNode, T anEntry)
{
   // Assertion: rootNode != null
   T result = null;
   int comparison = anEntry.compareTo(rootNode.getData());

   if (comparison == 0)
   {
      result = rootNode.getData();
      rootNode.setData(anEntry);
   }
   else if (comparison < 0)
   {
      if (rootNode.hasLeftChild())
         result = addEntry(rootNode.getLeftChild(), anEntry);
      else
         rootNode.setLeftChild(new BinaryNode<>(anEntry));
   }
   else
   {
     if (rootNode.hasRightChild())
         result = addEntry(rootNode.getRightChild(), anEntry);
      else
         rootNode.setRightChild(new BinaryNode<>(anEntry));
   } // end if

   return result;
} // end addEntry
```

Iterative Implementation

- Iterative algorithm for adding a new entry

```
Algorithm addEntry(binarySearchTree, anEntry)
// Adds a new entry to a binary search tree that is not empty.
// Returns null if anEntry did not exist already in the tree. Otherwise, returns the
// tree entry that matched and was replaced by anEntry.
result  = null
currentNode = root node of binarySearchTree found = false
while (found is false)
{
        if (anEntry matches the entry in currentNode)
        {
                found = true
                result = entry in currentNode
                Replace entry in currentNode with anEntry
        }
        else  if (newEntry  < entry in currentNode)
        {
```

Iterative Implementation

- Iterative algorithm for adding a new entry

```
                    if (currentNode has a left child)
                        currentNode = left child of currentNode
                    else
                    {
                        found = true
                        Give currentNode a left child containing anEntry
                    }
            } // end if-else
            else // anEntry > entry in currentNode
            {
                    if (currentNode has a right child)
                        currentNode = right child of currentNode
                    else
                    {
                        found = true
                        Give currentNode a right child containing anEntry
                    }
            }  // end if
    } // end while
    return result
```

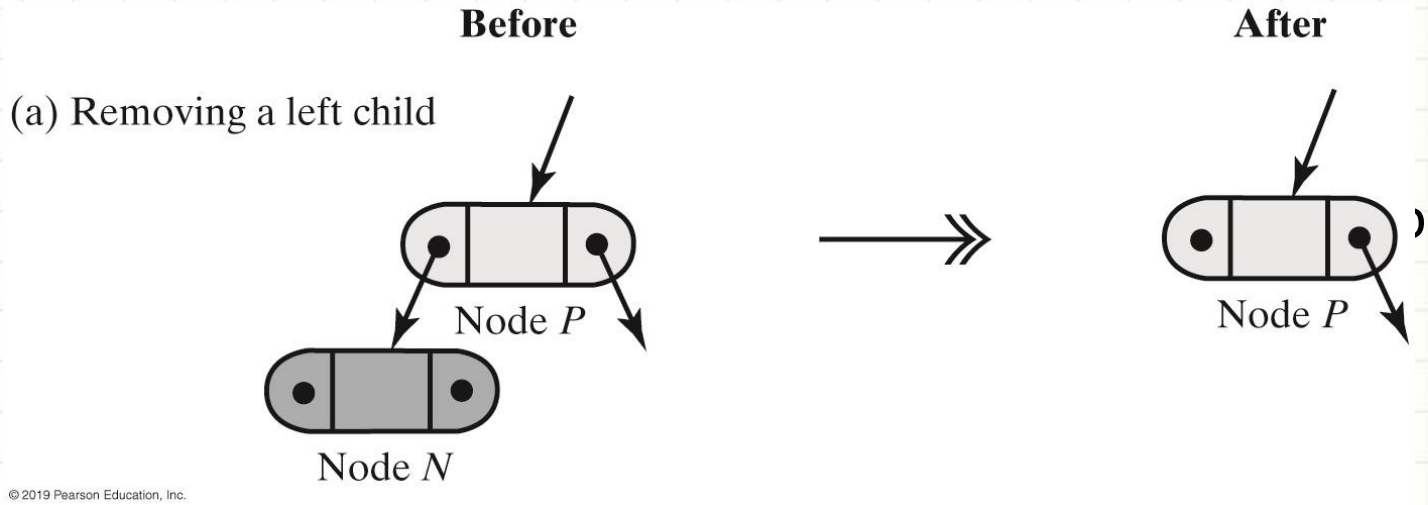
Iterative Implementation— method `addEntry`

```
private T addEntry(T anEntry) {
  BinaryNode<T> currentNode = getRootNode();
  // Assertion: currentNode != null
  T result = null;
  boolean found = false;

  while (!found)
  {
    T currentEntry = currentNode.getData();
    int comparison = anEntry.compareTo(currentEntry);

    if (comparison == 0)
    {  // anEntry matches currentEntry;
       // return and replace currentEntry
       found = true;
       result = currentEntry;
       currentNode.setData(anEntry);
    }
    else if (comparison < 0)
    {
       if (currentNode.hasLeftChild())
          currentNode = currentNode.getLeftChild();
       else
```

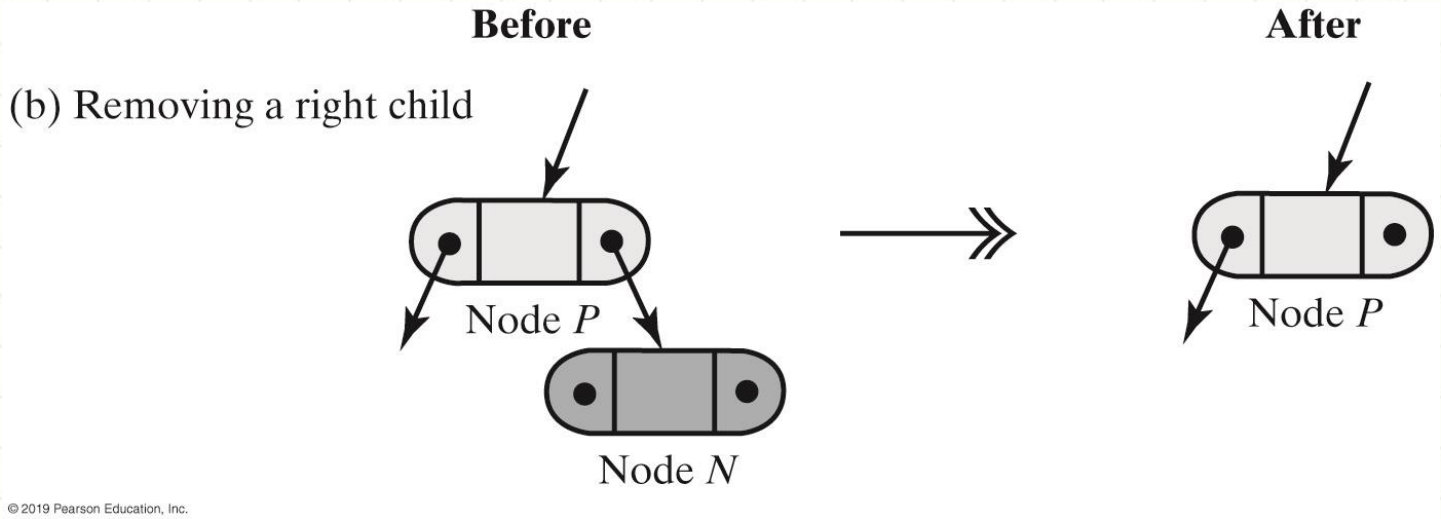
Iterative Implementation— method `addEntry`

```
    {
      // Assertion: comparison > 0

      if (currentNode.hasRightChild())
        currentNode = currentNode.getRightChild();
      else
      {
        found = true;
        currentNode.setRightChild(new BinaryNode<>(anEntry));
      } // end if
    } // end if
  } // end while

  return result;
} // end addEntry
```

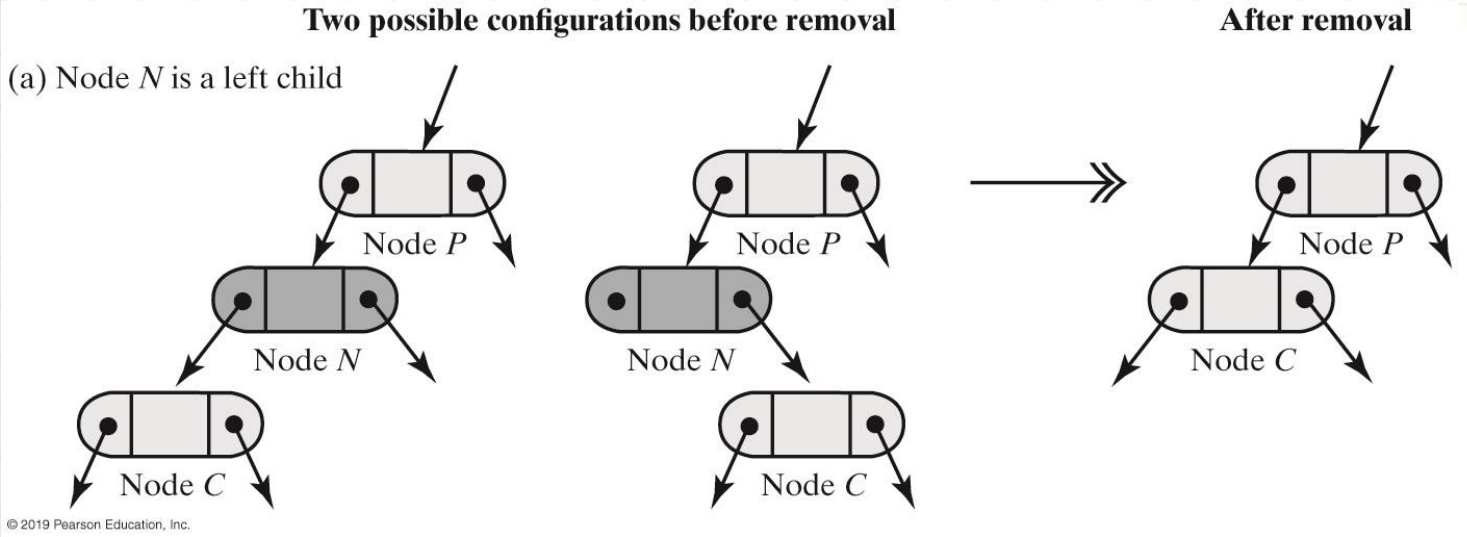
Removing a Value

- 

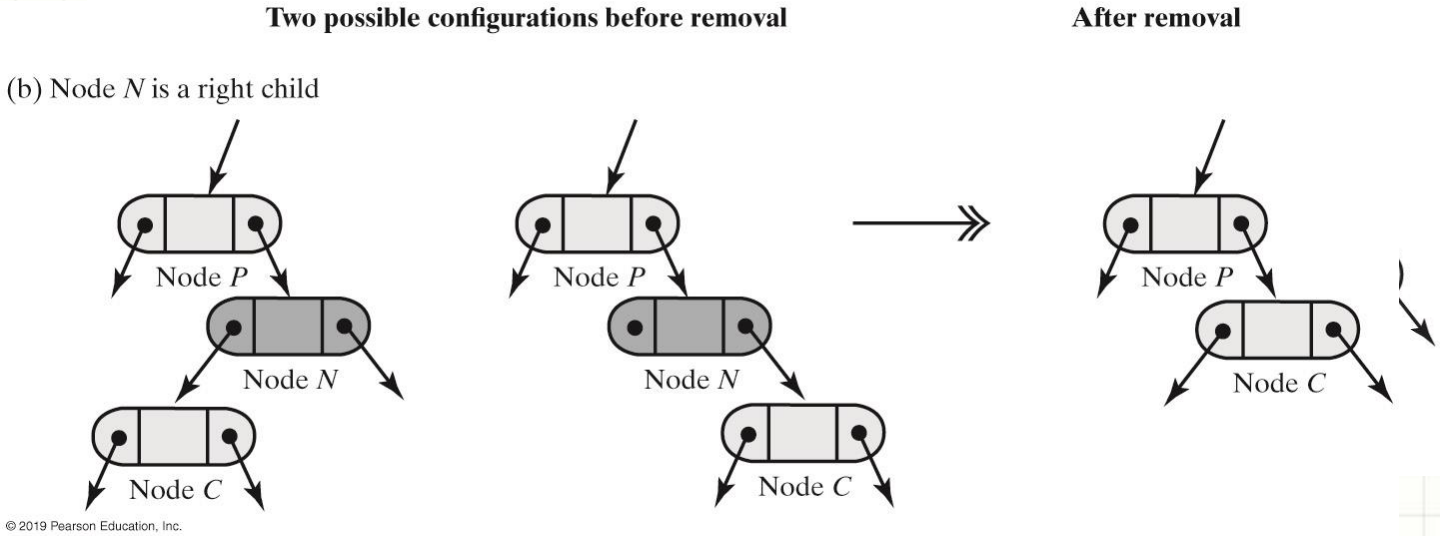
ode P



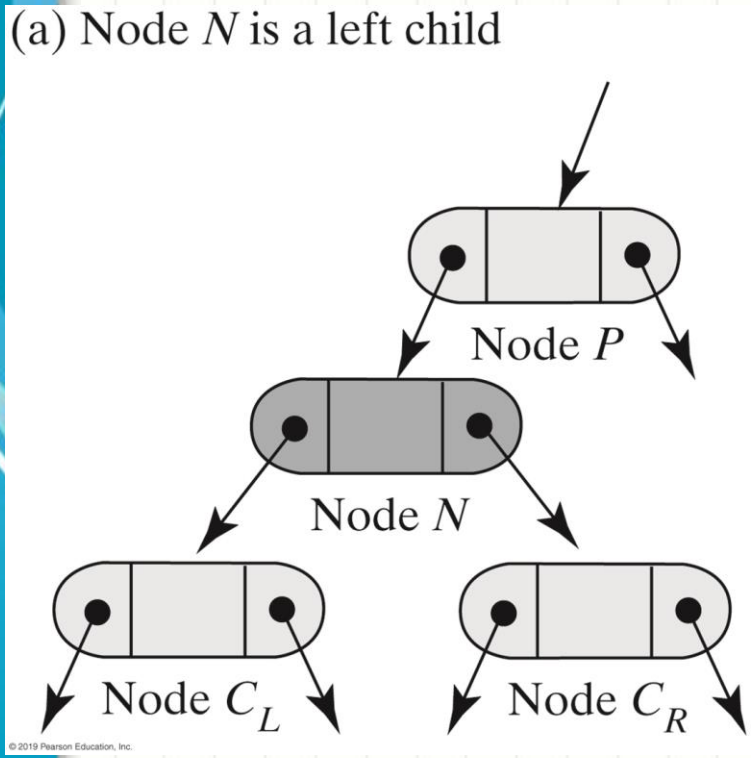
Removing a Value



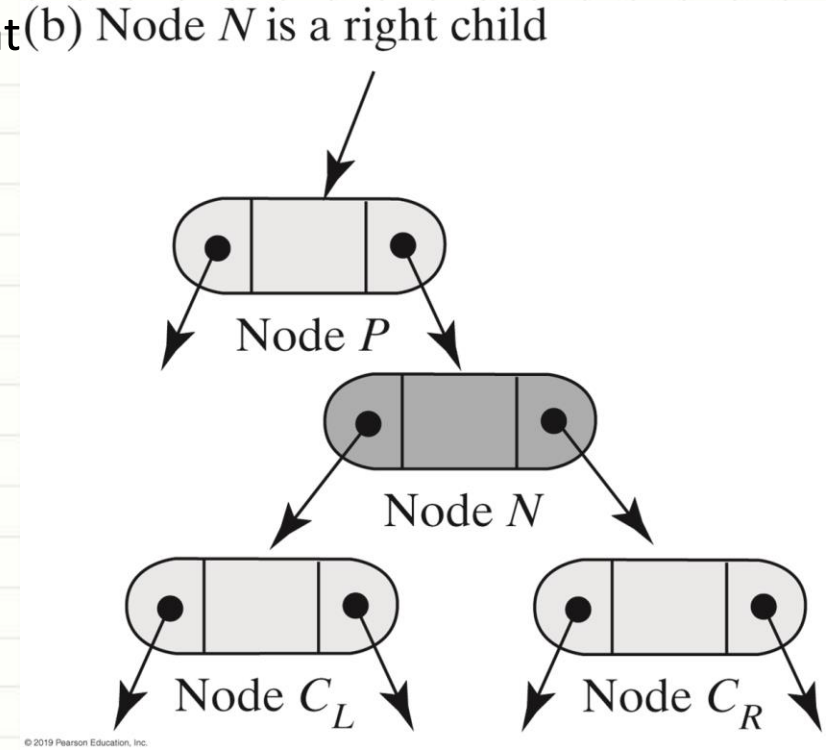
; one child



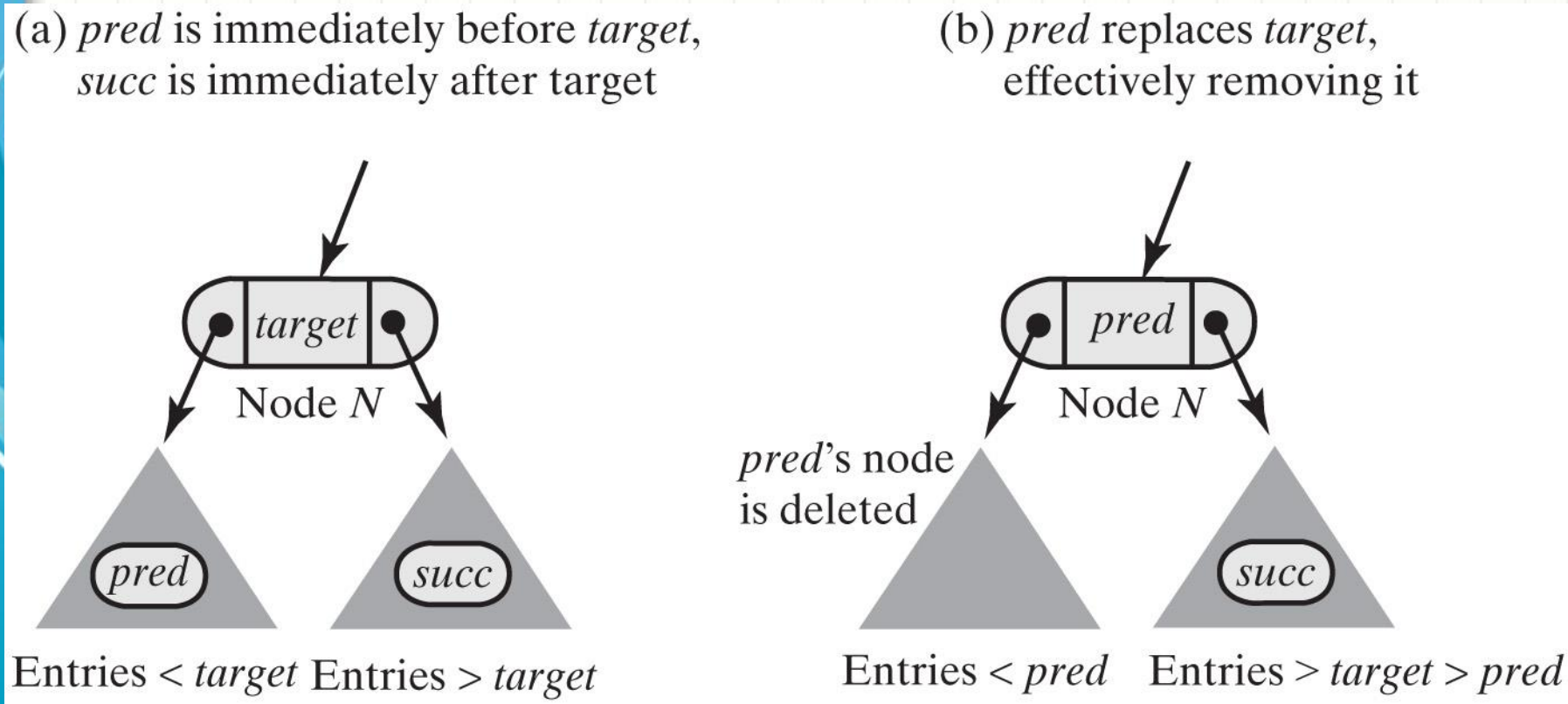
Removing a Value



configurat

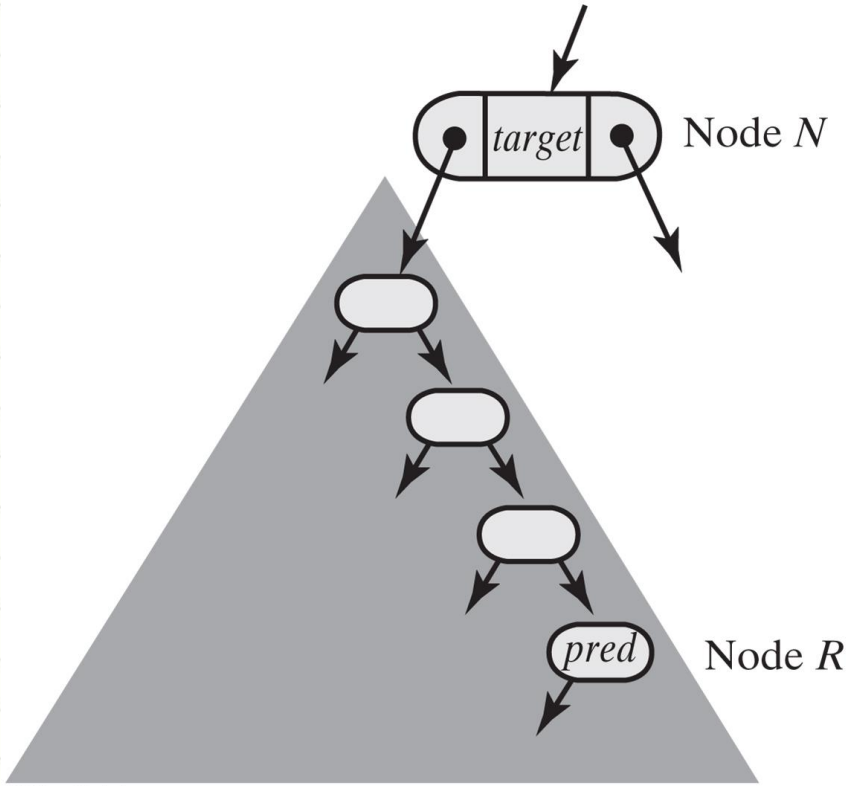


Removing a Value



© 2019 Pearson Education, Inc.

Removing a Value

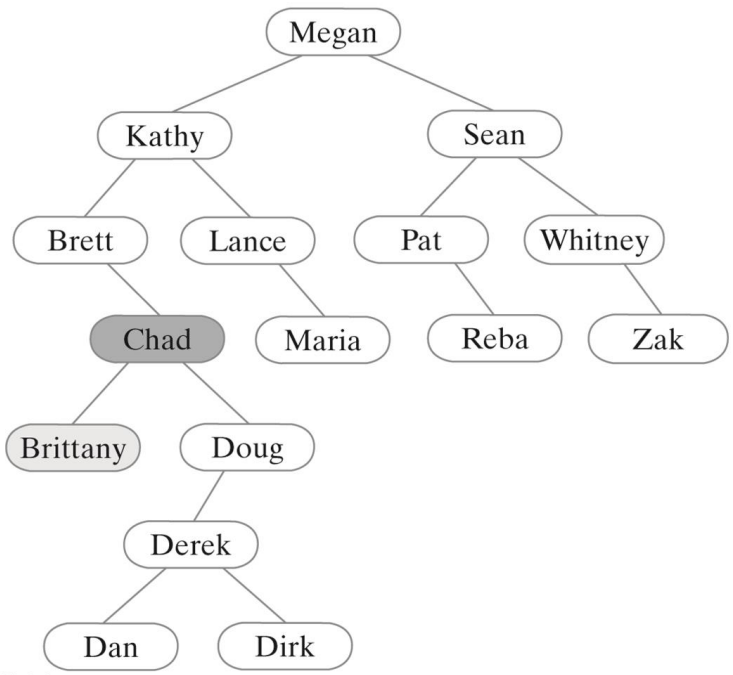


© 2019 Pearson Education, Inc.

- The largest entry *pred* in node *N's* left subtree occurs in the subtree's rightmost node *R*

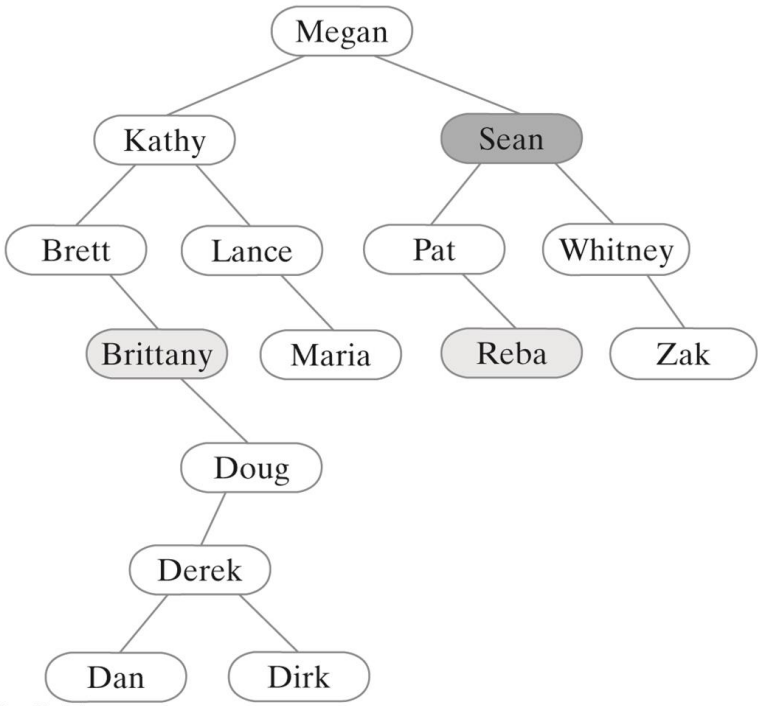
Successive removals from a binary search tree

(a) A binary search tree



© 2019 Pearson Education, Inc.

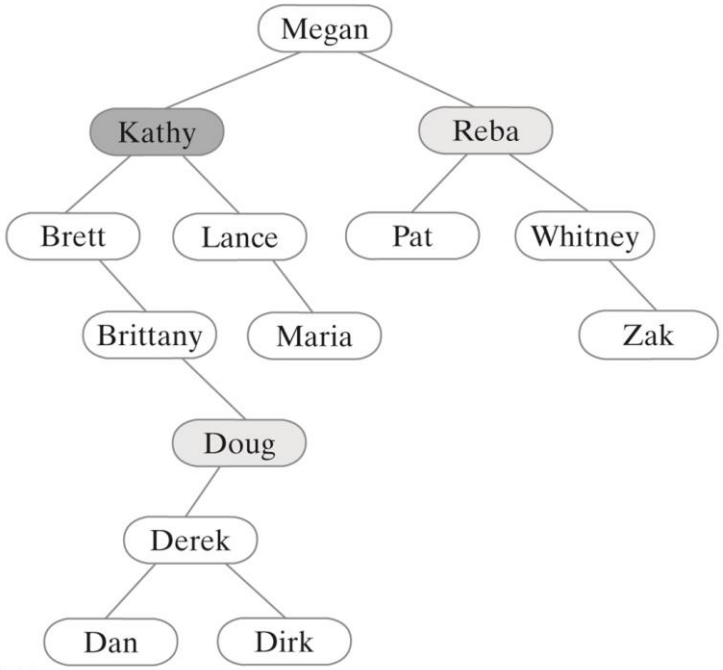
(b) The tree after removing *Chad*



© 2019 Pearson Education, Inc.

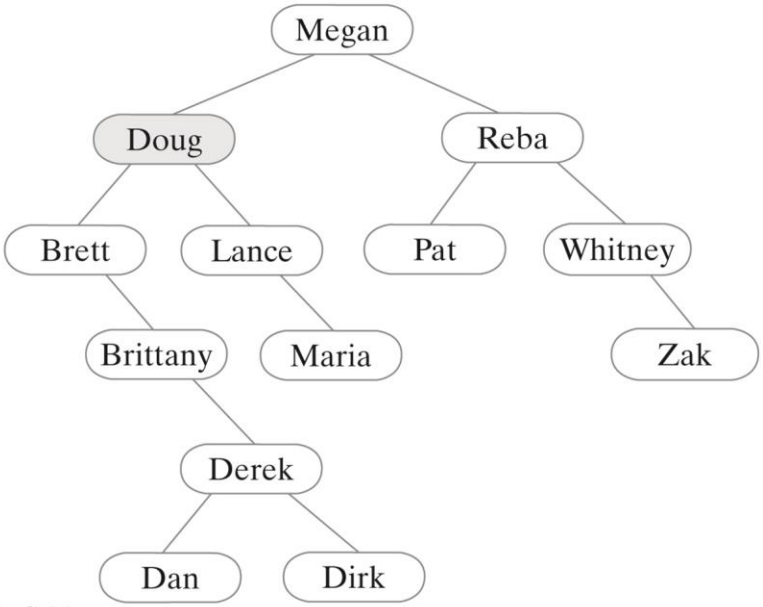
Successive removals from a binary search tree

(c) The tree after removing *Sean*



© 2019 Pearson Education, Inc.

(d) The tree after removing *Kathy*

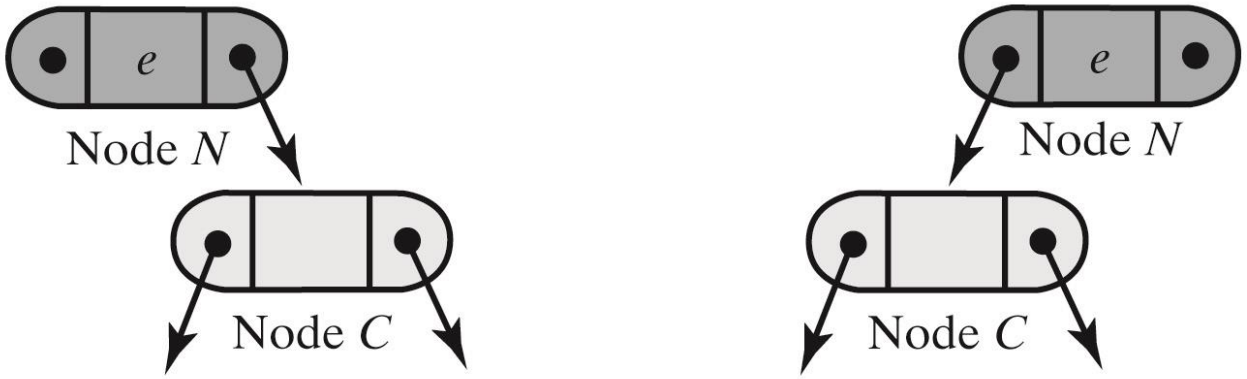


© 2019 Pearson Education, Inc.

Removing a Value

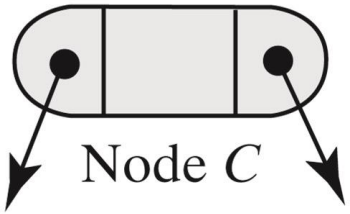
- F e child

(a) Two possible configurations of a tree's root with one child



© 2019 Pearson Education, Inc.

(b) The tree after removing its root



© 2019 Pearson Education, Inc.

Recursive Implementation

```
Algorithm remove(binarySearchTree, anEntry)
oldEntry = null
if (binarySearchTree is not empty)
{
        if (anEntry matches the entry in the root of binarySearchTree)
        {
        oldEntry = entry in root
        removeFromRoot(root of binarySearchTree)
        }
        else if (anEntry < entry in root)
                oldEntry = remove(left subtree of binarySearchTree, anEntr
        else // anEntry > entry in root
                oldEntry = remove(right subtree of binarySearchTree, anEn
}
return oldEntry
```

- Recursive algorithm describes the method's logic at a high level

Recursive Implementation

- The public method `remove`

```
public T remove(T anEntry)
{
   ReturnObject oldEntry = new ReturnObject(null);
   BinaryNode<T> newRoot = removeEntry(getRootNode(), anEntry, oldEntry);
   setRootNode(newRoot);

   return oldEntry.get();
} // end remove
```

Recursive Implementation

```
// Removes an entry from the tree rooted at a given node.
private BinaryNode<T> removeEntry(BinaryNode<T> rootNode, T anEntry,
                                  ReturnObject oldEntry)
{
   if (rootNode != null)
   {
      T rootData = rootNode.getData();
      int comparison = entry.compareTo(rootData);

      if (comparison == 0)       // anEntry == root entry
      {
         oldEntry.set(rootData);
         rootNode = removeFromRoot(rootNode);
      }
      else if (comparison < 0)   // anEntry < root entry
      {
         BinaryNode<T> leftChild = rootNode.getLeftChild();
         BinaryNode<T> subtreeRoot = removeEntry(leftChild, anEntry, oldEntry);
         rootNode.setLeftChild(subtreeRoot);
      }
      else                       // anEntry > root entry
      {
         BinaryNode<T> rightChild = rootNode.getRightChild();
         // A different way of coding than for left child:
         rootNode.setRightChild(removeEntry(rightChild, anEntry, oldEntry));
      } // end if
   } // end if

   return rootNode;
} // end removeEntry
```

- The private method `removeEntry`

Recursive Implementation

- The algorithm `removeFromRoot`

```
Algorithm removeFromRoot(rootNode)
// Removes the entry in a given root node of a subtree.
if (rootNode has two children)
{
        largestNode = node with the largest entry in the left subtree of rootNode
        Replace the entry in rootNode with the entry in largestNode
        Remove largestNode from the tree
}
else if (rootNode has a right child)
        rootNode = rootNode's right child
else
    rootNode = rootNode's left child // Possibly null
    // Assertion: If rootNode was a leaf, it is now null
return rootNode
```

Recursive Implementation

```
// Removes the entry in a given root node of a subtree.
private BinaryNode<T> removeFromRoot(BinaryNode<T> rootNode)
{
   // Case 1: rootNode has two children
   if (rootNode.hasLeftChild() && rootNode.hasRightChild())
   {
      // Find node with largest entry in left subtree
      BinaryNode<T> leftSubtreeRoot = rootNode.getLeftChild();
      BinaryNode<T> largestNode = findLargest(leftSubtreeRoot);

      // Replace entry in root
      rootNode.setData(largestNode.getData());

      // Remove node with largest entry in left subtree
      rootNode.setLeftChild(removeLargest(leftSubtreeRoot));
   } // end if

   // Case 2: rootNode has at most one child
   else if (rootNode.hasRightChild())
      rootNode = rootNode.getRightChild();
   else
      rootNode = rootNode.getLeftChild();

   // Assertion: If rootNode was a leaf, it is now null

   return rootNode;
} // end removeEntry
```

- The private method `removeFromRoot`

Recursive Implementation

- The private method `findLargest`

```
// Finds the node containing the largest entry in a given tree.
// rootNode is the root node of the tree.
// Returns the node containing the largest entry in the tree.
private BinaryNode<T> findLargest(BinaryNode<T> rootNode)
{
   if (rootNode.hasRightChild())
      rootNode = findLargest(rootNode.getRightChild());

   return rootNode;
} // end findLargest
```

Recursive Implementation

- The private method `removeLargest`

```
// Removes the node containing the largest entry in a given tree.
// rootNode is the root node of the tree.
// Returns the root node of the revised tree.
private BinaryNode<T> removeLargest(BinaryNode<T> rootNode)
{
   if (rootNode.hasRightChild())
   {
      BinaryNode<T> rightChild = rootNode.getRightChild();
      rightChild = removeLargest(rightChild);
      rootNode.setRightChild(rightChild);
   }
   else
      rootNode = rootNode.getLeftChild();

   return rootNode;
} // end removeLargest
```

Iterative Implementation

- Pseudocode that describes `remove`

```
Algorithm remove(anEntry)
result = null
currentNode = node that contains a match for anEntry
parentNode  =  currentNode's parent
if (currentNode != null) // That is, if entry is found
{
        result = currentNode's data (the anEntry to be removed from the tree)
        // Case 1
        if (currentNode has two children)
        {
                // Get node to remove and its parent
                nodeToRemove = node containing anEntry inorder predecessor; it has at most one c
                parentNode = nodeToRemove's parent
                Copy entry from nodeToRemove to currentNode
                currentNode = nodeToRemove
                // Assertion: currentNode is the node to be removed; it has at most one child
                // Assertion: Case 1 has been transformed to Case 2
        }
        //Case 2: currentNode has at most one child
        Delete currentNode from the tree
}
return result
```

Iterative `remove` Implementation

- The public method `remove`

```
public T remove(T entry)
{
   T result = null;

   // Locate node (and its parent) that contains a match for entry
   NodePair pair = findNode(entry);
   BinaryNode<T> currentNode = pair.getFirst();
   BinaryNode<T> parentNode = pair.getSecond();

   if (currentNode != null) // Entry is found
   {
      result = currentNode.getData(); // Get entry to be removed

      // Case 1: currentNode has two children
      if (currentNode.hasLeftChild() && currentNode.hasRightChild())
      {
         // Replace entry in currentNode with the entry in another node
         // that has at most one child; that node can be deleted

         // Get node to remove (contains inorder predecessor; has at
```

Iterative `remove` Implementation

- The public method `remove`

```
        // Copy entry from nodeToRemove to currentNode
        currentNode.setData(nodeToRemove.getData());

        currentNode = nodeToRemove;
        // Assertion: currentNode is the node to be removed; it has at
        //            most one child
        // Assertion: Case 1 has been transformed to Case 2
      } // end if

      // Case 2: currentNode has at most one child; delete it
      removeNode(currentNode, parentNode);
    } // end if

    return result;
  } // end remove
```

Iterative Implementation

- The private method `findNode`

```
private NodePair findNode(T entry)
{
   NodePair result = new NodePair();
   boolean found = false;

//  . . .

   if (found)
      result = new NodePair(currentNode, parentNode);
      // Located entry is currentNode.getData()

   return result;
} // end findNode
```

Iterative Implementation

- Pseudocode for the private method `getNodeToRemove`

```
// Find the in-order predecessor by searching the left subtree; it will be the largest
// entry in the subtree, occurring in the node as far right as possible
leftSubtreeRoot = left child of currentNode
rightChild = leftSubtreeRoot
priorNode = currentNode
while (rightChild has a right child)
{
    priorNode = rightChild
    rightChild = right child of rightChild
}
// Assertion: rightChild is the node to be removed and has no more than one child
```

Iterative `getNodeToRemove` Implementation

- Implementation of the private method `getNodeToRemove`

```
private NodePair getNodeToRemove(BinaryNode<T> currentNode)
{
   // Find node with largest entry in left subtree by
   // moving as far right in the subtree as possible
   BinaryNode<T> leftSubtreeRoot = currentNode.getLeftChild();
   BinaryNode<T> rightChild = leftSubtreeRoot;
   BinaryNode<T> priorNode = currentNode;

   while (rightChild.hasRightChild())
   {
      priorNode = rightChild;
      rightChild = rightChild.getRightChild();
   } // end while

   // rightChild contains the inorder predecessor and is the node to
   // remove; priorNode is its parent

   return new NodePair(rightChild, priorNode);
} // end getNodeToRemove
```

Iterative `removeNode` Implementation

- The private method `removeNode`

```
private void removeNode(BinaryNode<T> nodeToRemove,
                                                    BinaryNode<T> parentNode)
{
   BinaryNode<T> childNode;

   if (nodeToRemove.hasLeftChild())
      childNode = nodeToRemove.getLeftChild();
   else
      childNode = nodeToRemove.getRightChild();

   // Assertion: if nodeToRemove is a leaf, childNode is null

   if (nodeToRemove == getRootNode())
      setRootNode(childNode);
   else if (parentNode.getLeftChild() == nodeToRemove)
      parentNode.setLeftChild(childNode);
   else
      parentNode.setRightChild(childNode);
} // end removeNode
```

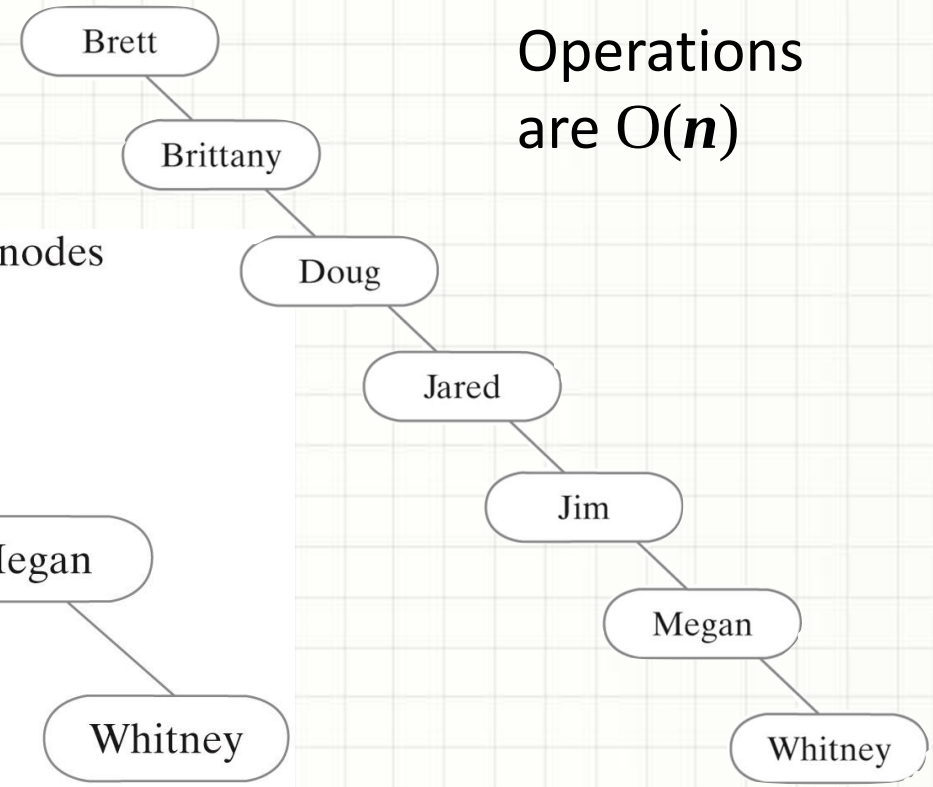
Efficiency of Operations

- For tree of height $\boldsymbol{h}$
  - The operations add, remove, and **`getEntry`** are $\mathrm{O}(\boldsymbol{h})$
- If tree of $\boldsymbol{n}$ nodes has height $\boldsymbol{h} = \boldsymbol{n}$
  - These operations are $\mathrm{O}(\boldsymbol{n})$
- Shortest tree is full
  - Results in these operations being $\mathrm{O}(\log \boldsymbol{n})$

Efficiency of Operations

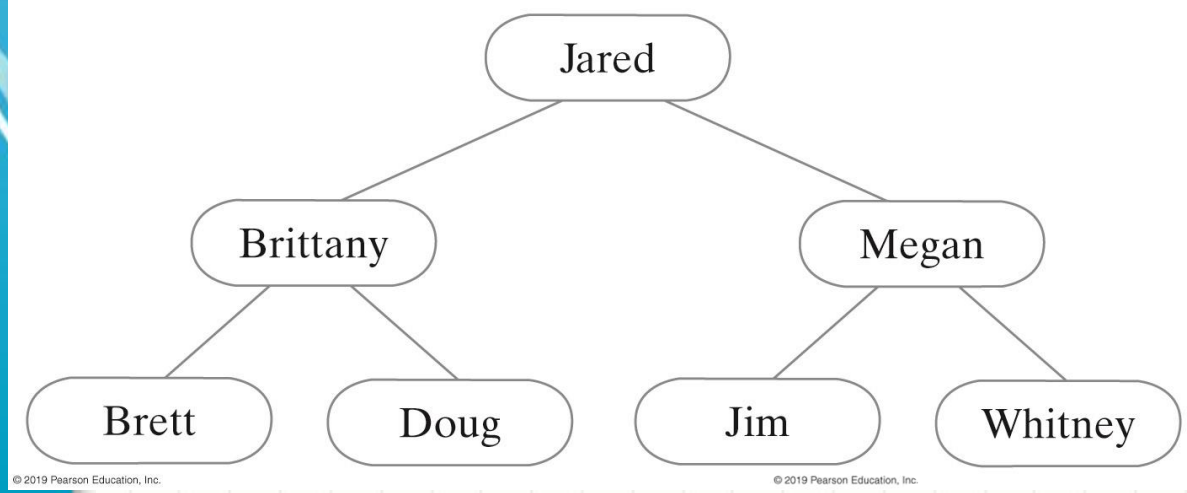
- Two binary search trees that contain the same data

(b) The tallest binary search tree having seven nodes



Operations are $O(\boldsymbol{n})$

(a) The shortest binary search tree having seven nodes



© 2019 Pearson Education, Inc.

© 2019 Pearson Education, Inc.

Operations are $O(\log \boldsymbol{n})$

Heap and Maxheap

- Heap
  - ***Complete binary tree*** whose nodes contain Comparable objects
- Maxheap
  - Object in each node is greater than or equal to the objects in the node's descendants

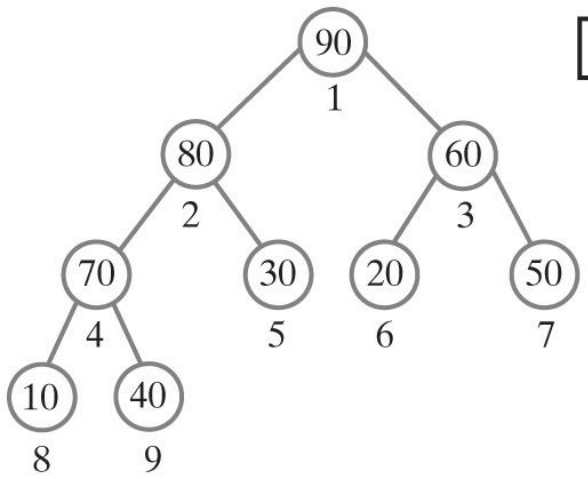
Heap and Maxheap

- Interface for the ADT Maxheap

```
/** An interface for the ADT maxheap. */
public interface MaxHeapInterface<T extends Comparable<? super T>>
{  // See Segment 24.33 for a commented version.
   public void add(T newEntry);
   public T removeMax();
   public T getMax();
   public boolean isEmpty();
   public int getSize();
   public void clear();
} // end MaxHeapInterface
```

An Array to Represent a Heap

- Use an array to represent a complete binary tree
- Number nodes in the order in which a level-order traversal would visit them
- Can locate either the children or the parent of any node
  - Perform a simple computation on the node's number



	90	80	60	70	30	20	50	10	40			
0	1	2	3	4	5	6	7	8	9	10	11	12

© 2019 Pearson Education, Inc.

An Array to Represent a Heap

- The class MaxHeap, partially completed

```
/** A class that implements the ADT maxheap by using an array. */
public final class MaxHeap<T extends Comparable<? super T>>
            implements MaxHeapInterface<T>
{
   private T[] heap;      // Array of heap entries; ignore heap[0]
   private int lastIndex; // Index of last entry and number of entries
   private boolean integrityOK = false;
     private static final int DEFAULT_CAPACITY = 25;
     private static final int MAX_CAPACITY = 10000;

  public MaxHeap(int initialCapacity)
   {
      // Is initialCapacity too small?
      if (initialCapacity < DEFAULT_CAPACITY)
         initialCapacity = DEFAULT_CAPACITY;
      else // Is initialCapacity too big?
         checkCapacity(initialCapacity);

      // The cast is safe because the new array contains null entries
      @SuppressWarnings("unchecked")
      T[] tempHeap = (T[])new Comparable[initialCapacity + 1];
```

An Array to Represent a Heap

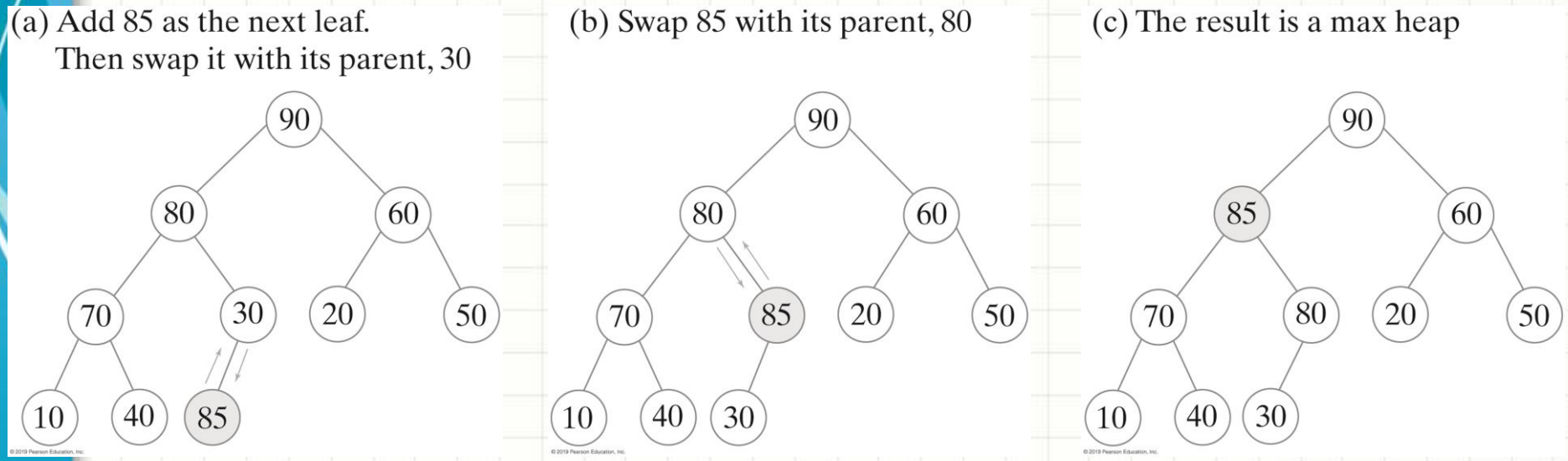
```
public MaxHeap()
{
   this(DEFAULT_CAPACITY); // Call next constructor
} // end default constructor

public T getMax()
{
    checkIntegrity();
   T root = null;
   if (!isEmpty())
      root = heap[1];
   return root;
} // end getMax

public boolean isEmpty()
{
   return lastIndex < 1;
} // end isEmpty

public int getSize()
{
   return lastIndex;
} // end getSize
```

An Array to Represent a Heap

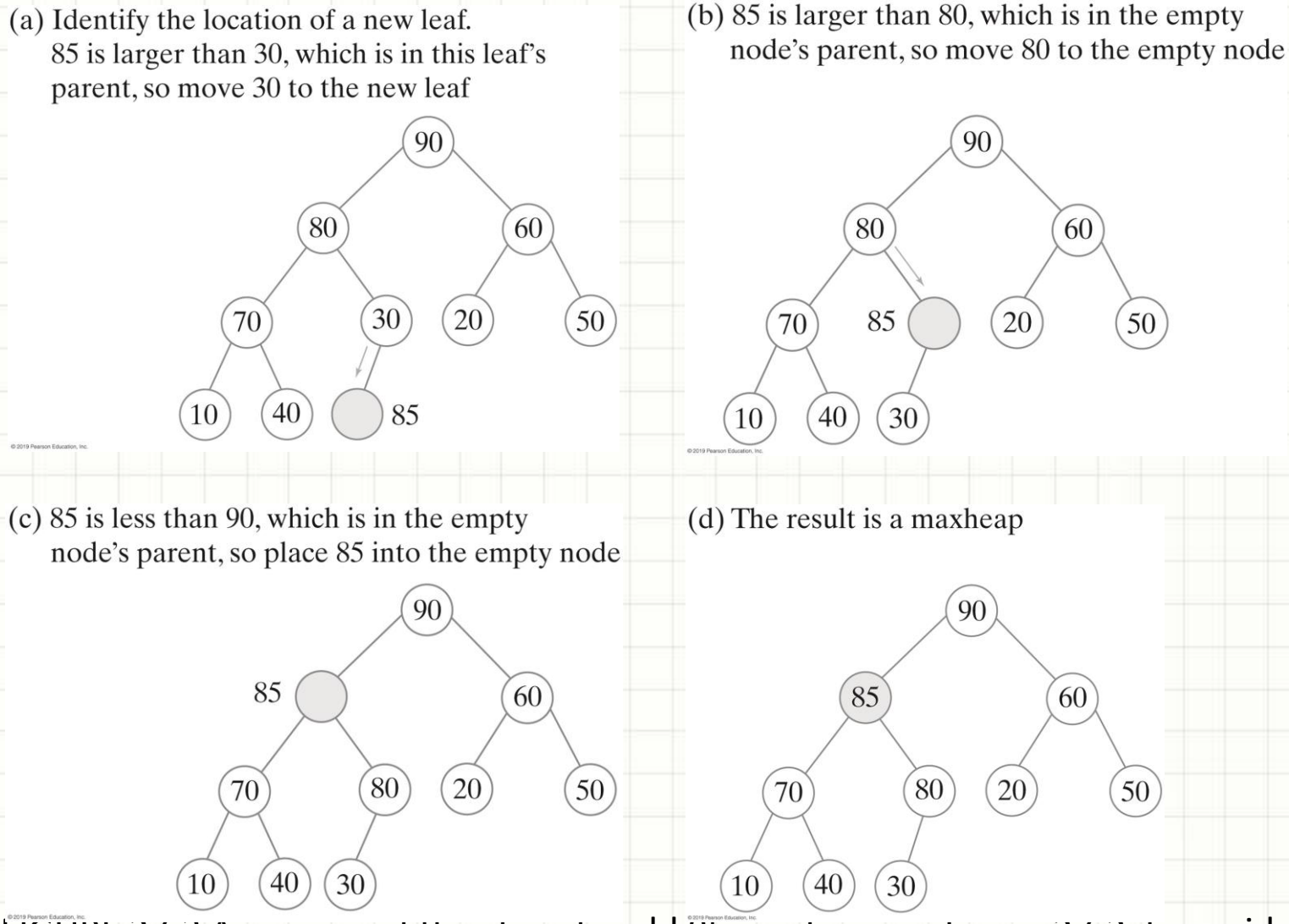
```
public void add(T newEntry)
{
// Will address later — See Segment 27.8.
} // end add

public T removeMax()
{
// Will address later — See Segment 27.12.
} // end removeMax

public void clear()
{
  checkIntegrity();
  while (lastIndex > -1)
  {
    heap[lastIndex] = null;
    lastIndex--;
  } // end while
  lastIndex = 0;
} // end clear

// Private methods
```

Adding an Entry

- The steps in adding 85 to the maxheap

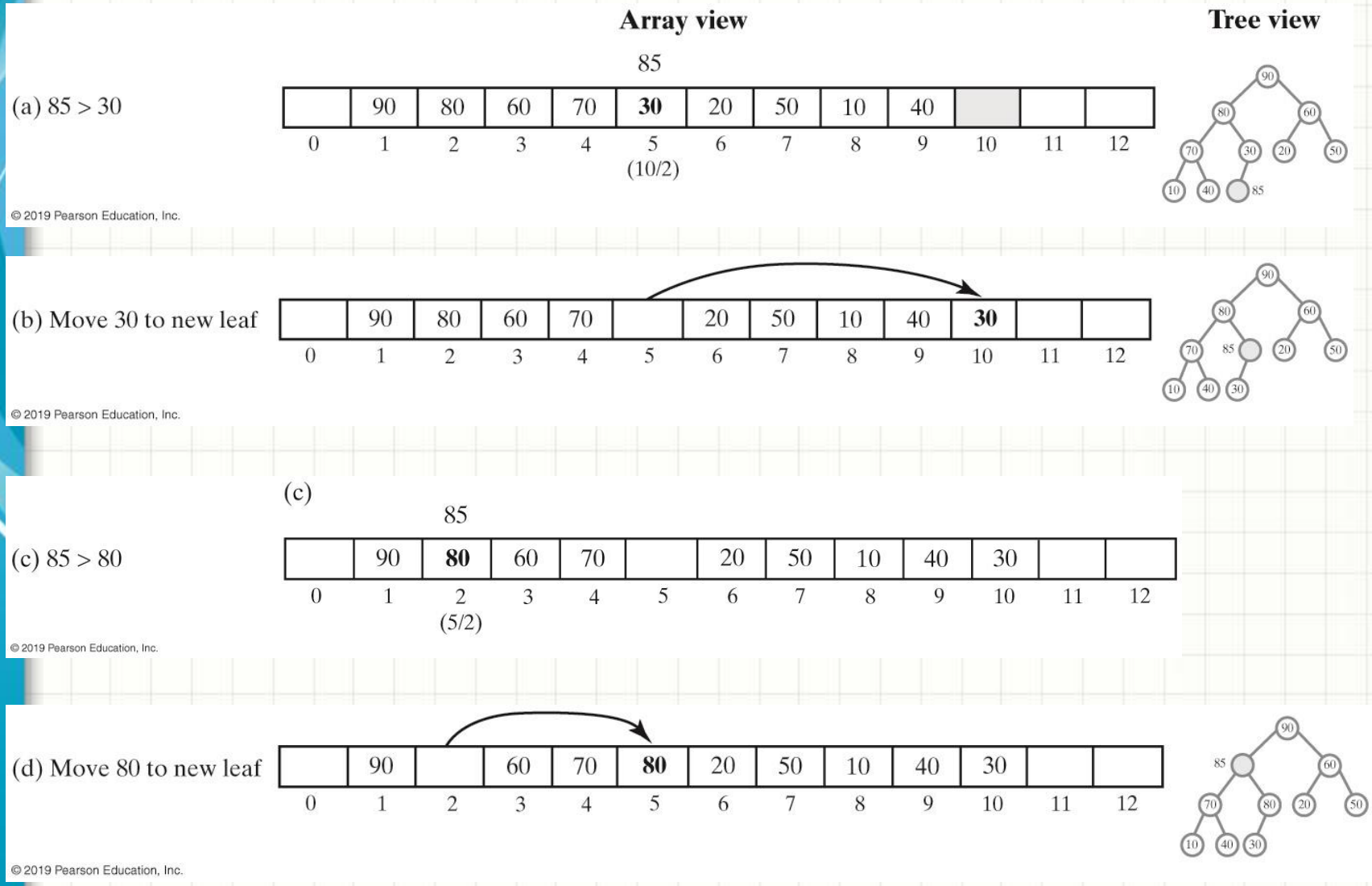


Adding an Entry without Swaps



- FIGURE 27-3 A revision of the steps to add 85, as shown in Figure 27-2, to avoid swaps

Adding an Entry to Heap

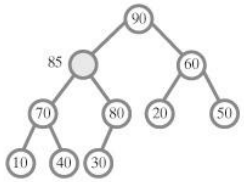


- FIGURE 27-4 An array representation of the steps in Figure 27-3

Adding an Entry to Heap

(d) Move 80 to new leaf

0	1	2	3	4	5	6	7	8	9	10	11	12
	90		60	70	**80**	20	50	10	40	30		



© 2019 Pearson Education, Inc.

(e) 85 < 90

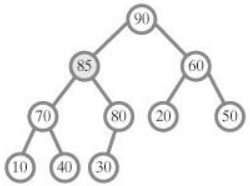
85

0	1 (2/2)	2	3	4	5	6	7	8	9	10	11	12
	90		60	70	80	20	50	10	40	30		

© 2019 Pearson Education, Inc.

(f) Insert 85 into vacancy

0	1	2	3	4	5	6	7	8	9	10	11	12
	90	**85**	60	70	80	20	50	10	40	30		



© 2019 Pearson Education, Inc.

- An array representation of the steps

Adding an Entry

- Algorithm to add a new entry to a heap

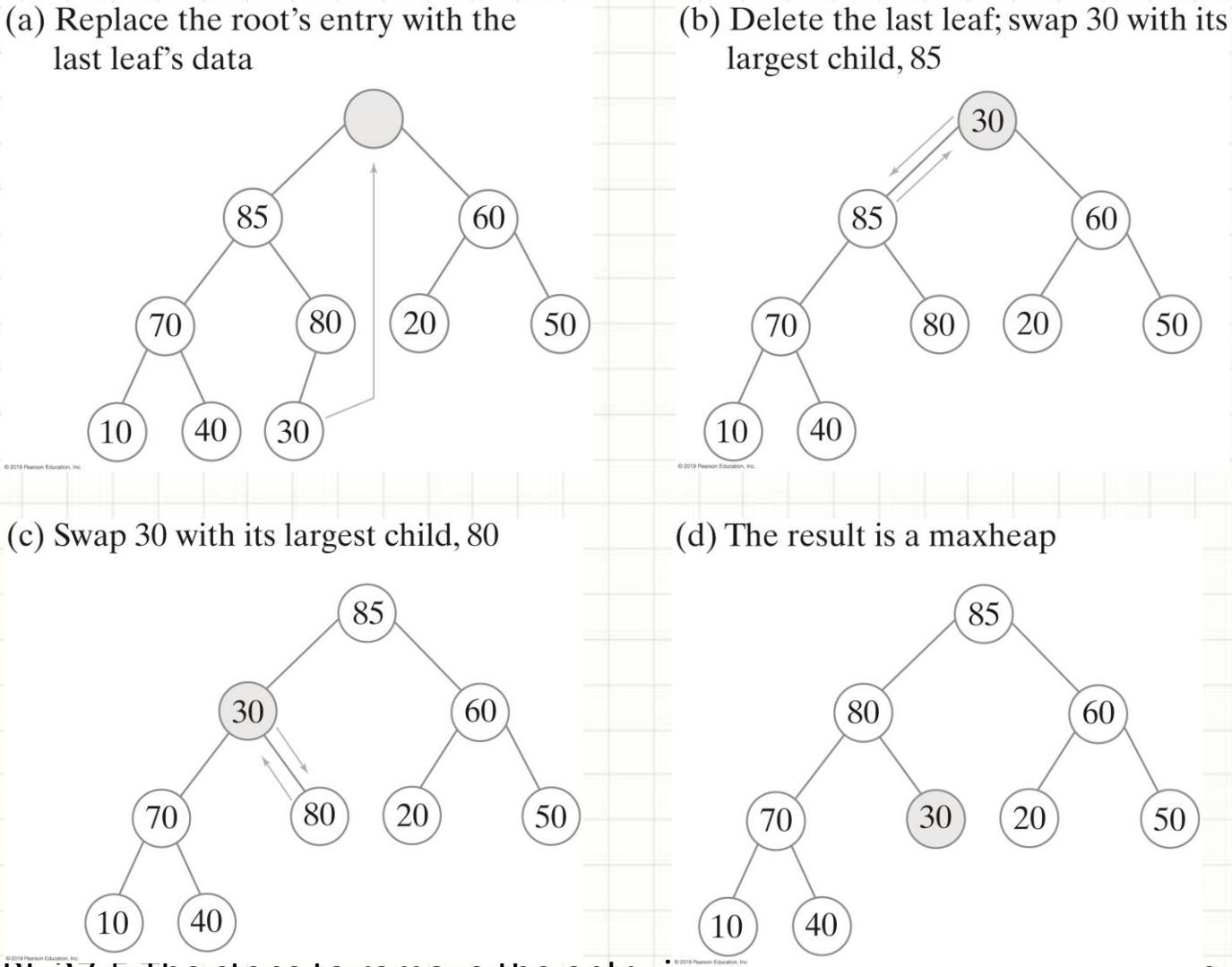
```
Algorithm add(newEntry)
// Precondition: The array heap has room for another entry.
newIndex = index of next available array location
parentIndex  =  newIndex/2  //Index of parent of available location
while (parentIndex > 0 and newEntry > heap[parentIndex])
{
        heap[newIndex] = heap[parentIndex] // Move parent to available location
        // Update indices
        newIndex = parentIndex parentIndex = newIndex/2
}
heap[newIndex] = newEntry          // Place new entry in correct location
if (the array heap is full)
        Double the size of the array
```

Adding an Entry

- The method `add`

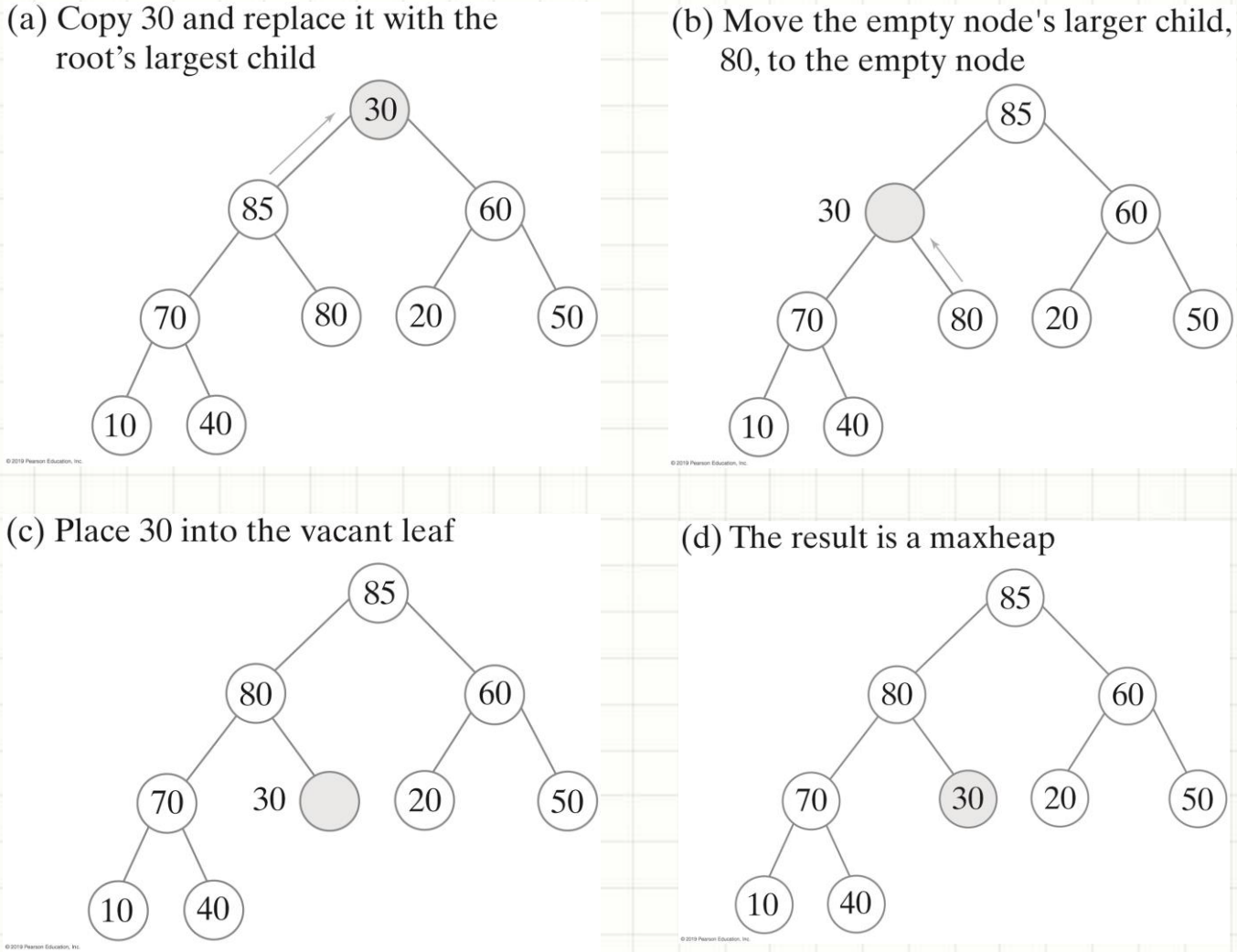
```
public void add(T newEntry)
{
   checkIntegrity();        // Ensure initialization of data fields
   int newIndex = lastIndex + 1;
   int parentIndex = newIndex / 2;
   while ( (parentIndex > 0) && newEntry.compareTo(heap[parentIndex]) > 0)
   {
      heap[newIndex] = heap[parentIndex];
      newIndex = parentIndex;
      parentIndex = newIndex / 2;
   } // end while

   heap[newIndex] = newEntry;
   lastIndex++;
   ensureCapacity();
} // end add
```

Removing a Value from A Heap



- FIGURE 27-5 The steps to remove the entry in the root of the maxheap in Figure 27-3d

Removing a Value without Swaps



- FIGURE 27-6 The steps to transform the semiheap in Figure 27-5b into a heap without using swaps

Removing the Root

- Algorithm to transform a semiheap to a heap

```
Algorithm reheap(rootIndex)
// Transforms the semiheap rooted at rootIndex into a heap
done = false
orphan = heap[rootIndex]
while (!done and heap[rootIndex] has a child)
{
   largerChildIndex = index of the larger child of heap[rootIndex]
   if (orphan < heap[largerChildIndex])
   {
      heap[rootIndex] = heap[largerChildIndex]
      rootIndex = largerChildIndex
   }
   else
      done = true
}
heap[rootIndex] = orphan
```

Removing the Root

```
private void reheap(int rootIndex)
{
   boolean done = false;
   T orphan = heap[rootIndex];
   int leftChildIndex = 2 * rootIndex;

   while (!done && (leftChildIndex <= lastIndex) )
   {
      int largerChildIndex = leftChildIndex; // Assume larger
      int rightChildIndex = leftChildIndex + 1;

      if ( (rightChildIndex <= lastIndex) &&
            heap[rightChildIndex].compareTo(heap[largerChildIndex]) > 0)
      {
         largerChildIndex = rightChildIndex;
      } // end if

      if (orphan.compareTo(heap[largerChildIndex]) < 0)
      {
         heap[rootIndex] = heap[largerChildIndex];
         rootIndex = largerChildIndex;
         leftChildIndex = 2 * rootIndex;
      }
      else
         done = true;
   } // end while
  heap[rootIndex] = orphan;
} // end reheap
```

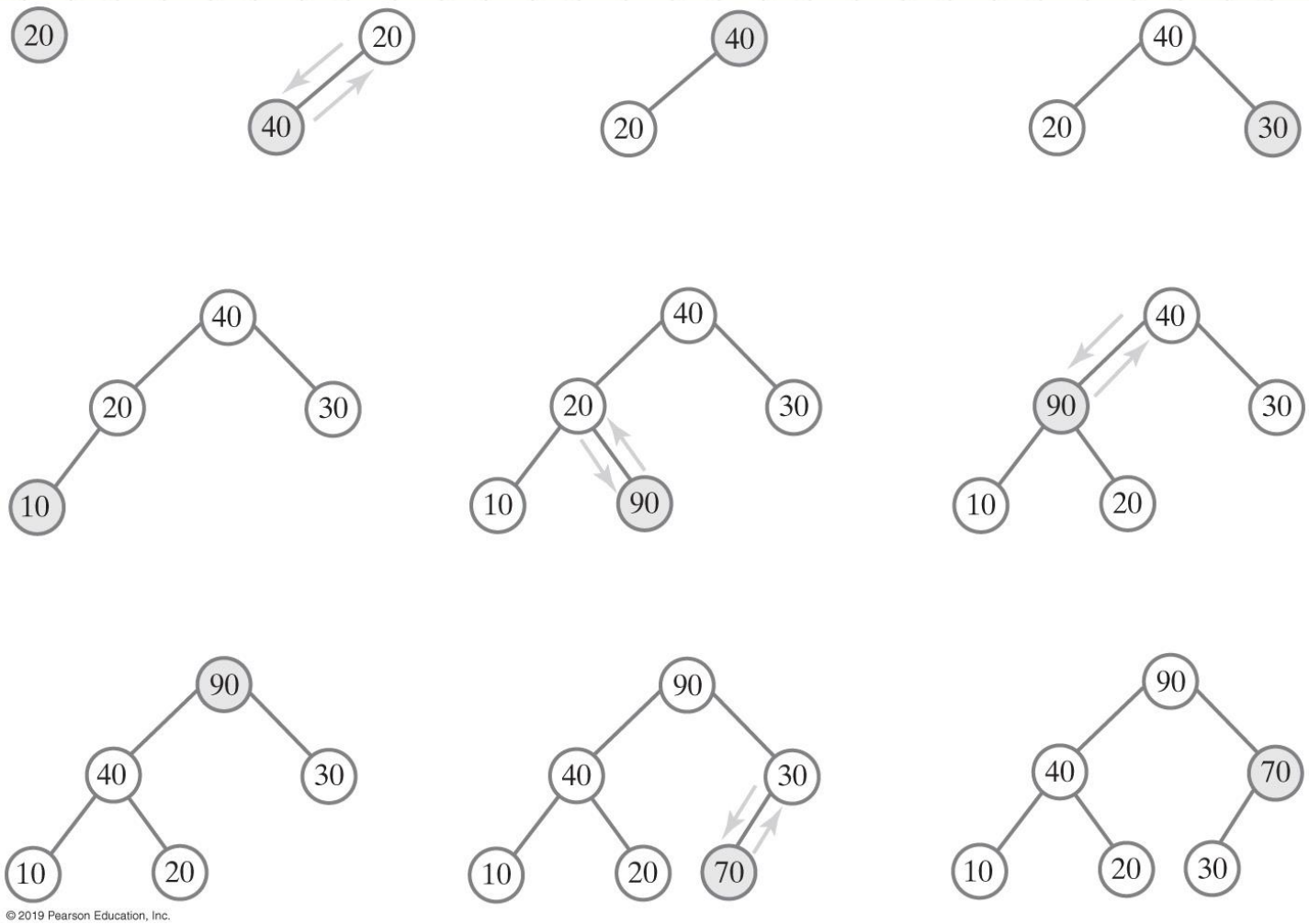
- Implementation of the reheap algorithm as a private method

Removing the Root

- Implementation of the `removeMax` method

```
public T removeMax()
{
   checkIntegrity();            // Ensure initialization of data fields
   T root = null;

   if (!isEmpty())
   {
      root = heap[1];           // Return value
      heap[1] = heap[lastIndex]; // Form a semiheap
      lastIndex--;              // Decrease size
      reheap(1);                // Transform to a heap
   } // end if

   return root;
} // end removeMax
```

Creating a Heap



© 2019 Pearson Education, Inc.

- FIGURE 27-7 The steps in adding 20, 40, 30, 10, 90, and 70 to an initially empty heap

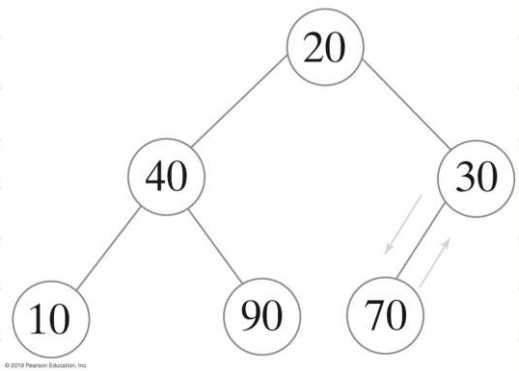
Creating a Heap

(a) An array of entries

	20	40	30	10	90	70
0	1	2	3	4	5	6

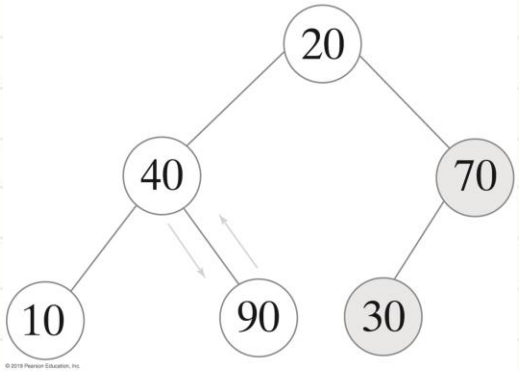
© 2019 Pearson Education, Inc.

(b) The complete tree that the array represents



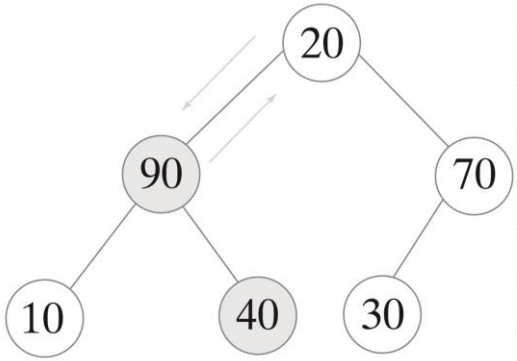
© 2019 Pearson Education, Inc.

(c) After `reheap(3)`



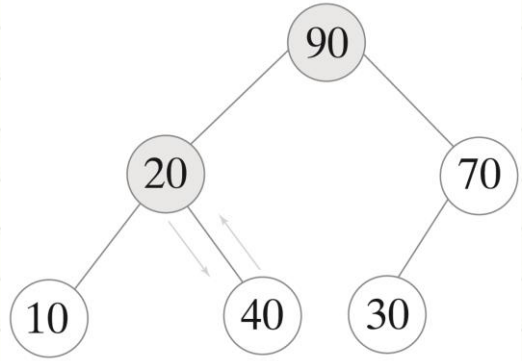
© 2019 Pearson Education, Inc.

(d) After `reheap(2)`



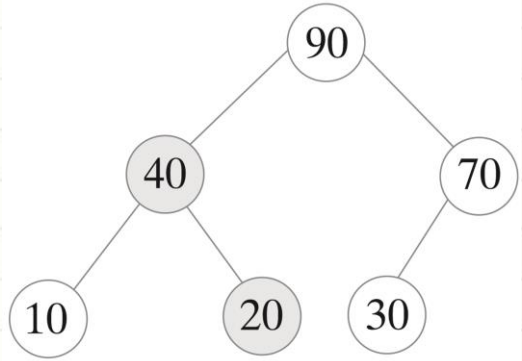
© 2019 Pearson Education, Inc.

(e) During `reheap(1)`



© 2019 Pearson Education, Inc.

(f) After `reheap(1)`



© 2019 Pearson Education, Inc.

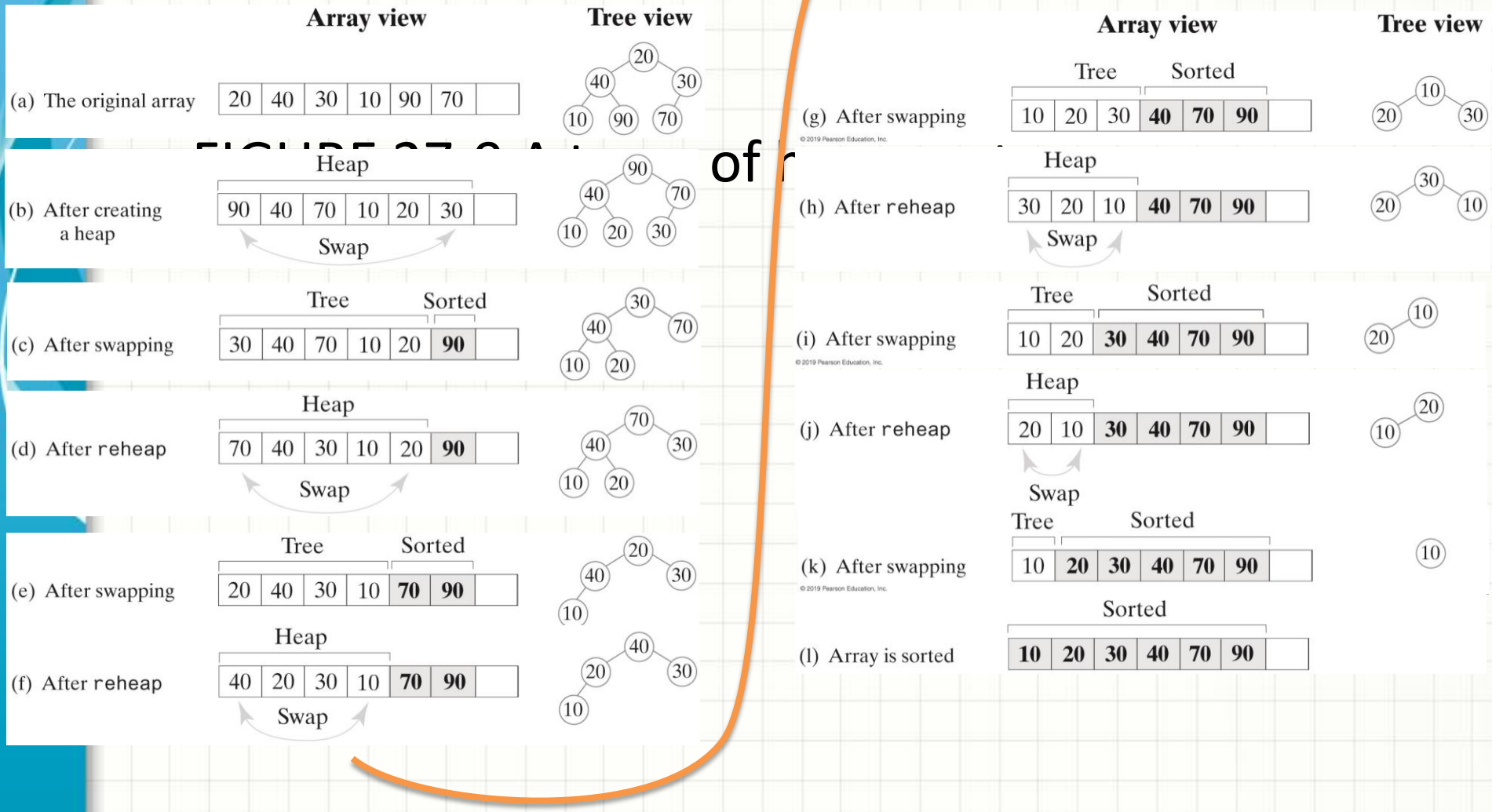
- The steps in creating a heap of the entries 20, 40, 30, 10, 90, and 70 by using `reheap`

Creating a Heap

- Another constructor for the class `MaxHeap`

```
public MaxHeap(T[] entries)
{
   this(entries.length); // Call other constructor
   lastIndex = entries.length;
   // Assertion: integrityOK = true

   // Copy given array to data field
   for (int index = 0; index < entries.length; index++)
      heap[index + 1] = entries[index];

   // Create heap
   for (int rootIndex = lastIndex / 2; rootIndex > 0; rootIndex--)
      reheap(rootIndex);
} // end constructor
```

Heap Sort


Array view
Tree view
(a) The original array
20 40 30 10 90 70
(b) After creating a heap
Heap
90 40 70 10 20 30
Swap
(c) After swapping
Tree
Sorted
30 40 70 10 20 90
(d) After reheap
Heap
70 40 30 10 20 90
Swap
(e) After swapping
Tree
Sorted
20 40 30 10 70 90
(f) After reheap
Heap
40 20 30 10 70 90
Swap
(g) After swapping
Tree
Sorted
10 20 30 40 70 90
(h) After reheap
Heap
30 20 10 40 70 90
Swap
(i) After swapping
Tree
Sorted
10 20 30 40 70 90
(j) After reheap
Heap
20 10 30 40 70 90
Swap
(k) After swapping
Tree
Sorted
10 20 30 40 70 90
(l) Array is sorted
Sorted
10 20 30 40 70 90

Heap Sort - Revised `reheap` Method

```
private static <T extends Comparable<? super T>>
      void reheap(T[] heap, int rootIndex, int lastIndex)
{
   boolean done = false;
   T orphan = heap[rootIndex];
   int leftChildIndex = 2 * rootIndex + 1;

   while (!done && (leftChildIndex <= lastIndex))
   {
      int largerChildIndex = leftChildIndex;
      int rightChildIndex = leftChildIndex + 1;

      if ( (rightChildIndex <= lastIndex) &&
            heap[rightChildIndex].compareTo(heap[largerChildIndex]) > 0)
      {
         largerChildIndex = rightChildIndex;
      } // end if

      if (orphan.compareTo(heap[largerChildIndex]) < 0)
      {
         heap[rootIndex] = heap[largerChildIndex];
         rootIndex = largerChildIndex;
         leftChildIndex = 2 * rootIndex + 1;
      }
      else
         done = true;
   } // end while
  heap[rootIndex] = orphan;
} // end reheap
```

Heap Sort

- The `heapSort` method with time efficiency is O(*n* log *n*)

```
public static <T extends Comparable<? super T>>
      void heapSort(T[] array, int n)
{
   // Create first heap
   for (int rootIndex = n / 2 - 1; rootIndex >= 0; rootIndex--)
      reheap(array, rootIndex, n - 1);

   swap(array, 0, n - 1);

   for (int lastIndex = n - 2; lastIndex > 0; lastIndex--)
   {
      reheap(array, 0, lastIndex);
      swap(array, 0, lastIndex);
   } // end for
} // end heapSort
```